МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»
Факультет Программной Инженерии и Компьютерной Техники

ЛАБОРАТОРНАЯ РАБОТА №3

по дисциплине

«Сервис-ориентированная архитектура»

Вариант: 888

Работу выполнили:
Студенты группы P3411
Болорболд Аригуун
Орчиков Даниил Валерьевич

Лектор:
Цопа Евгений Алексеевич
Практик:
Кривоносов Егор Дмитриевич

г. Санкт-Петербург
2025

Содержание

1. Текст задания 3
2. Выполнение 4
   2.1. Исходный код 4
   2.2. Проброс портов: 4
   2.3. Настройка Service Discovery 4
   2.4. Настройка микросервисов 6
   2.5. Пул EJB 9
3. Вывод 10

1. Текст задания

Переработать веб-сервисы из лабораторной работы #2 таким образом, чтобы они реализовывали основные концепции микросервисной архитектуры. Для этого внести в оба сервиса — «вызываемый» и «вызывающий» перечисленные ниже изменения.

Изменения в «вызываемом» сервисе:

- Разделить приложение на два модуля — веб-приложение с веб-сервисом и `EJB-jar` с бизнес-компонентами.
- Переместить всю логику из класса сервиса в `Stateless EJB`. В классе сервиса оставить только обращение к методам бизнес-интерфейса. `EJB`-компонент должен быть доступен удалённо (иметь `Remote`-интерфейс).
- Сформировать на уровне сервера приложений пул компонентов `EJB` настраиваемой мощности, динамически расширяемый при увеличении нагрузки.
- Установить ПО `Consul` и настроить `Service Discovery` с его помощью. Сервис должен регистрироваться в `Service Discovery` в момент запуска.

Изменения в «вызывающем» сервисе:

- Переработать сервис, обеспечив возможность его развёртывания на платформе `Spring Cloud`.
- Использовать все основные компоненты архитектуры `Spring Cloud`: `Config Service`, `Load Balancer`, `Service Discovery`.
- В качестве `Service Discovery` использовать `Eureka`.
- В качестве балансировщика нагрузки использовать `Ribbon`.
- Все запросы к сервисам должны проксироваться через `API Gateway`. В качестве `API Gateway` использовать `Zuul Proxy`.

Оба веб-сервиса и клиентское приложение должны сохранить полную совместимость с API, реализованными в рамках предыдущих лабораторных работ.

2. Выполнение

2.1. Исходный код

GitHub: https://github.com/17starplatinum/SOAJerseyClub

2.2. Проброс портов:

```
ssh -p 2222 s367452@se.ifmo.ru -L 15478:helios:15478 -L
15479:helios:15479
```

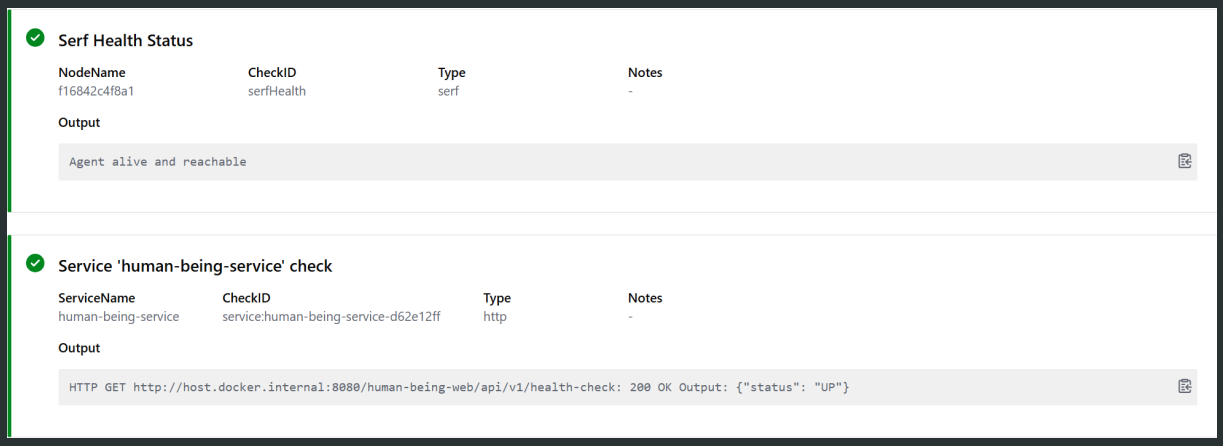
2.3. Настройка Service Discovery

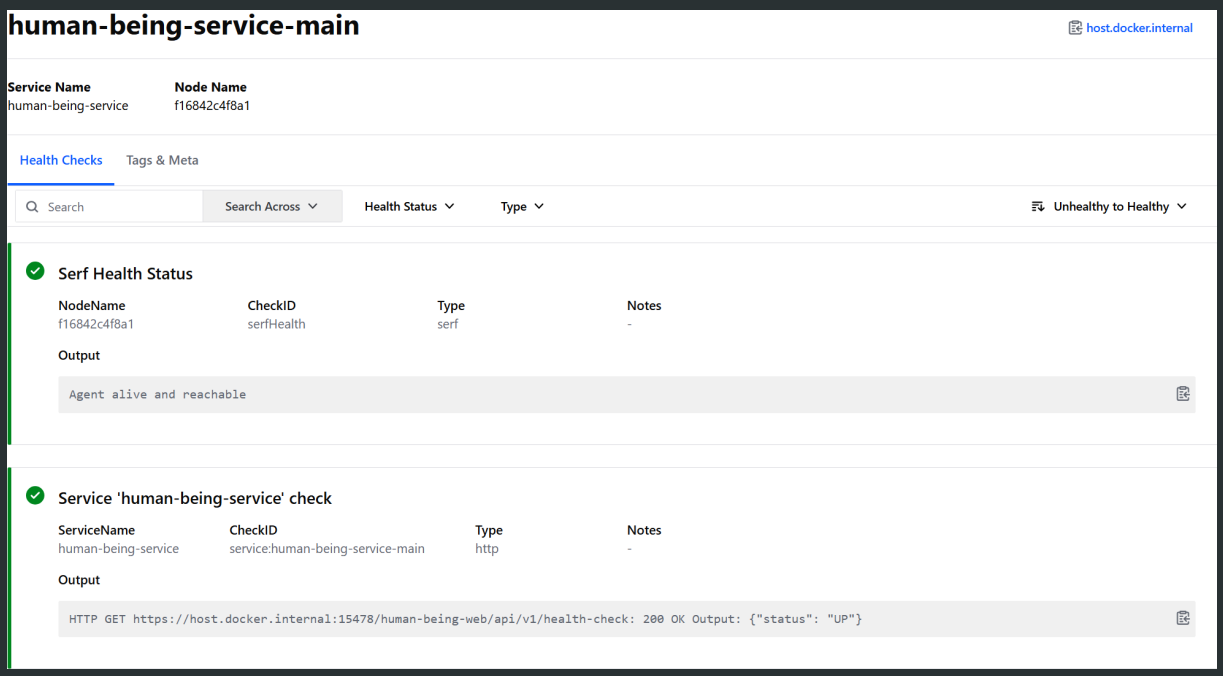
2.3.1. JAX-RS

1. ConsulRegistrar.java содержит методы, которые зарегистрирует и дерегистрирует сервис
2. Поднять экземпляр из докера:

```
consul:
  env_file: .env
  image: hashicorp/consul:1.22
  container_name: consul
  restart: unless-stopped
  environment:
    - CONSUL_HOST=${CONSUL_HOST}
    - CONSUL_PORT=${CONSUL_PORT}
    - CONSUL_DNS_PORT=${CONSUL_DNS_PORT}
  ports:
    - "${CONSUL_PORT}:${CONSUL_PORT}"
    - "${CONSUL_DNS_PORT}:${CONSUL_DNS_PORT}/udp"
  command: "agent -server -bootstrap-expect=1 -ui -client=0.0.0.0 -
bind=0.0.0.0 -datacenter=dc1"
```

3. Если смотреть из окна, то видно следующее:



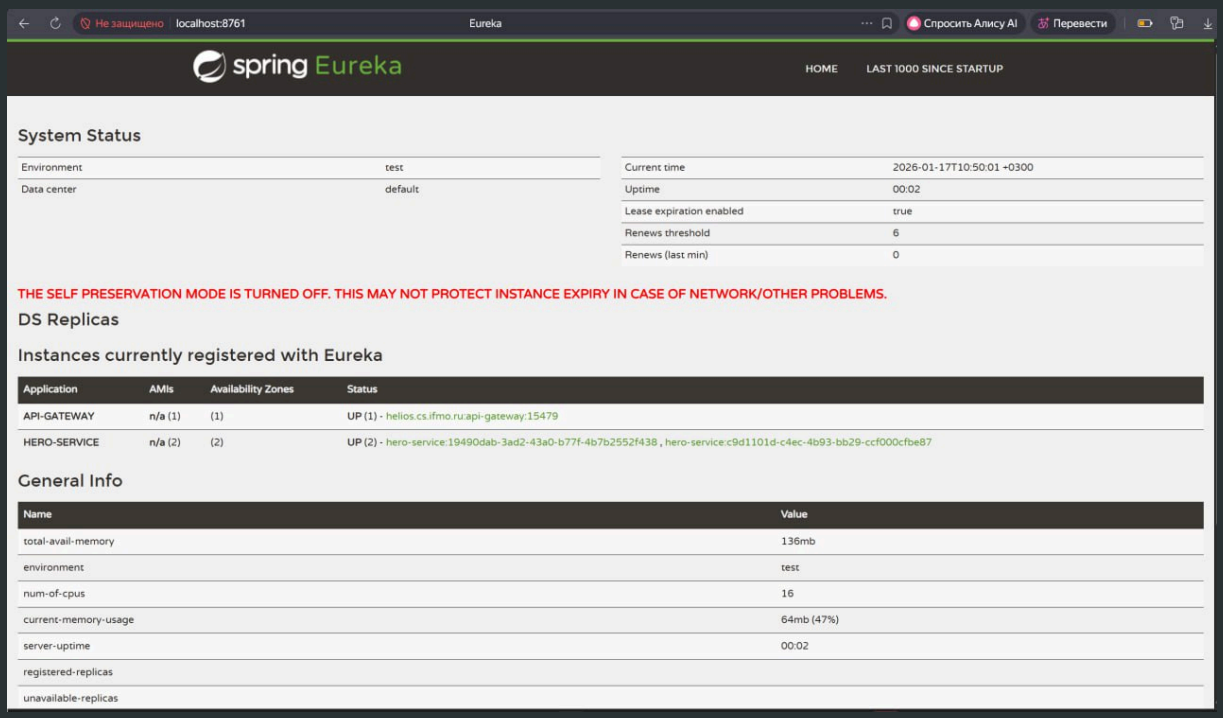


что подтверждает работу `Service Discovery`.

2.3.2. Spring

Сервер: `@EnableEurekaServer`, отключена self-preservation Клиенты:

- `@EnableDiscoveryClient`, указан defaultZone: `https://localhost:8761/eureka`
- Используется двусторонний TLS (HTTPS) для всех соединений
- Клиенты регистрируются со случайным портом (server.port: 0)



2.4. Настройка микросервисов

2.4.1. JAX-RS

Само приложение теперь состоит из 2 модулей: EJB и Web. Web отвечает за принятие запросов, а EJB - за бизнес-логику. EJB содержит в себе 6 отдельных бинов, которые отвечают за отдельные действия.

1. Выделил логику в имплементации Stateless EJB, а вызов оставил в самом бине.

```
@Remote
public interface AddHumanBeingRemote {
    Result<HumanBeingResponse> addHumanBeing(HumanBeingRequest
requestDto);
}
```

Логика есть в реализации, и логика сохранилась полностью.

```
@Stateless
@Transactional
public class AddHumanBeingBean implements AddHumanBeingRemote {
    @Inject
    private HumanBeingRepository humanBeingRepository;

    @Inject
    private HumanBeingMapper humanBeingMapper;

    @Inject
    private HeroServiceClient heroServiceClient;

    @Override
    public Result<HumanBeingResponse>
addHumanBeing(HumanBeingRequest requestDto) {
      // логика
    }
}
```

2. Поместил вызов этих бинов в адаптере, в human-being-web (отдельном модуле):

```
@ApplicationScoped
public class HumanBeingAdapter {
    @EJB(lookup = "java:global/human-being-ejb/AddHumanBeingBean!
ru.itmo.cs.dandadan.ejb.remote.AddHumanBeingRemote")
    private AddHumanBeingRemote addHumanBeingService;
    // и т.д.
}
```

Бины вызываются через `JNDI`. которые определяются ЖЦ `WildFly`:

```
13:52:30,509 INFO  [org.jboss.as.ejb3.deployment] (MSC service
thread 1-9) WFLYEJB0473: JNDI bindings for session bean named
'AddHumanBeingBean' in deployment unit 'deployment "human-being-
ejb.jar"' are as follows:

  java:global/human-being-ejb/AddHumanBeingBean!
ru.itmo.cs.dandadan.ejb.remote.AddHumanBeingRemote
 ...
```

2.4.2. Spring

Система построена на основе микросервисной архитектуры с использованием Spring Cloud и состоит из четырех основных сервисов:

- api-gateway (порт 15479) — единая точка входа (Zuul)
- config-server (порт 8888) — централизованный сервер конфигурации
- discovery-service (порт 8761) — сервис регистрации и обнаружения на `Eureka`
- hero-service — бизнес-сервис

Взаимодействие сервисов:

- При запуске сервисы получают конфигурацию с Config Server через HTTPS
- Все сервисы (кроме Config Server) регистрируются в Eureka
- Внешние клиенты обращаются только через API Gateway
- Gateway использует Eureka для обнаружения сервисов и балансировки нагрузки

Настройка Spring Cloud:

Config Server/Client: Сервер: `@EnableConfigServer`, native-режим (файлы из repo-config/) Клиенты: `spring-cloud-starter-config`, настройки в bootstrap.yml

API Gateway (Zuul):

- `@EnableZuulProxy`
- Маршрутизация: `/api/v1/** → hero-service`
- Интеграция с Eureka для service discovery

Балансировка нагрузки:

- Используется `Ribbon` через `@LoadBalanced RestTemplate`
- Автоматическое обнаружение инстансов через `Eureka`

Структура repo-config: application.yml:

```yaml
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/studs
    username: ${SPRING_DATASOURCE_USERNAME}
    password: ${SPRING_DATASOURCE_PASSWORD}
    driver-class-name: org.postgresql.Driver
  jpa:
    hibernate:
      ddl-auto: update
    properties:
      hibernate.dialect: org.hibernate.dialect.PostgreSQLDialect
    show-sql: true
```

hero-service.yml:

```yaml
spring:
  application:
    name: hero-service

cors:
  allowed-origins: http://localhost:5173, https://se.ifmo.ru
```

api-gateway.yml:

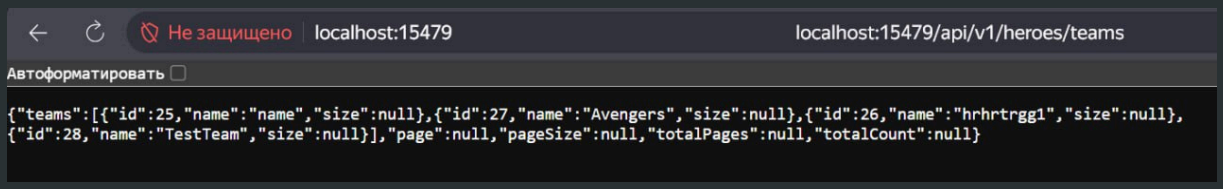
```yaml
spring:
  application:
    name: api-gateway

server:
  port: 15479
  ssl:
    enabled: true
    key-store: classpath:keystore/api-gateway.p12
    key-store-password: changeit
    key-passowrd: changeit
    kay-store-type: PKCS12
    key-alias: api-gateway
    trust-store: classpath:keystore/truststore.p12
    trust-store-password: changeit
    trust-store-type: PKCS12
    client-auth: need
```

eureka.yml:

```yaml
eureka:
  client:
    service-url:
      defaultZone: https://localhost:8761/eureka
```

```
    tls:
      enabled: true
      key-stire: classpath:keystore/api-gateway.p12
      key-store-password: changeit
      key-password: changeit
      key-store-type: PKCS12
      trust-store: classpath:keystore/truststore.p12
      trust-store-password: changeit
      trust-store-type: PKCS12

zuul:
  routes:
    hero-service:
      path: /api/v1/**
      serviceId: hero-service
      stripPrefix: true
      sensitivve-headers: Cookie, Set-Cookie

  ignored-headers: Access-Control-Allow-Credentials, Access-
Control-Allow-Origin
  host:
    connect-timeout-millis: 60000
    socket-timeout-millis: 60000

cors:
  allowed-origins: https://se.ifmo.ru, http://localhost:5173

ribbon:
  is-secure: true

hero-service:
  ribbon:
    IsSecure: true
```

Пример запроса через API Gateway:



2.5. Пул EJB

Оно делалось следующим путем:

```
[standalone@localhost:9990 /] /subsystem=ejb3/strict-max-bean-
instance-pool=human-being-service-pool:add(derive-size=from-worker-
```

```
pools, timeout=5, timeout-unit=MINUTES)
{"outcome" => "success"}

[standalone@localhost:9990 /] /subsystem=ejb3:write-
attribute(name=default-slsb-instance-pool, value=human-being-
service-pool)
{"outcome" => "success"}

[standalone@localhost:9990 /] reload
[standalone@localhost:9990 /] /subsystem=ejb3/strict-max-bean-
instance-pool=human-being-service-pool:read-resource
{
    "outcome" => "success",
    "result" => {
        "derive-size" => "from-worker-pools",
        "max-pool-size" => 20,
        "timeout" => 5L,
        "timeout-unit" => "MINUTES"
    }
}
```

3. Вывод

В задании на самом деле не рассматривается полноценная МСА, поэтому в рамках этой работы мы реализовали лишь концепции микросервисной архитектуры на своих существующих сервисах. Для того, чтобы новые сервисы взаимодействовали между собой внутри модули, а также с другими из другой модули, нам обеим пришлось настроить `Service Discovery` разными путями: через `Consul` и `Eureka`. Также здесь мы реализовали некоторые из популярных методов для обеспечения высокой доступности сервисов через пула бинов, балансировщика нагрузки и шлюз API.

Сдали первую лабу по СОА в октябре
Сдача второй лабы в декабре
imgflip.com